



AI CODING FOR VFX ARTISTS

Stack Guide

What to use when you start building: local tools, DCC tools, desktop apps and web apps. Plus API keys, storage and domains.

John Huikku · A preview of Class 2 · checked September 2026

Why a "stack"?

A **stack** is just the set of tools a project is built with, like a pipeline. You don't need to understand every piece. Telling your assistant *which* stack to use gets you a well-organized result it can keep working on, instead of something different every time.

How to use this guide: find what you're building in the table, then say it in your prompt: *"Build this with Python, uv and Gradio."* The assistant handles the rest. Read the sections below when you want to know why.

Which stack for what

Building...	Use
A quick utility: rename, convert, organize	Python + uv + ffmpeg
A utility with a simple UI	Python + Gradio
A tool inside a DCC	That app's Python + PySide6, or an MCP
A desktop app for your team	Tauri + the web UI stack, or PySide6 + PyInstaller
A web app	React · Vite · TypeScript · Tailwind · shadcn/ui, hosted on GitHub Pages, Netlify or Vercel
...that uses an API key	+ a serverless function or a Railway backend
...that needs logins or a database	+ Supabase
...that stores lots of media	+ Backblaze B2 or Cloudflare R2

Pick services an agent can set up for you. Everything recommended here has a command-line tool, an API or an MCP, so the assistant can configure it instead of you clicking through dashboards.

1. Local tools (most of your first projects)

Piece	What it does
Python	The scripting language VFX apps use.
uv	Installs Python and keeps each tool's add-ons separate, so installing one thing never breaks another.

Piece	What it does
ffmpeg / ffprobe	Converts video; reports clip specs.
Pillow, OpenCV	Image handling: thumbnails, grids, simple processing.
OpenImageIO / OpenEXR	Reading and writing EXRs and image sequences properly.
watchdog	Watches a folder and acts when new files land ("make a review copy of anything dropped here").
Typer	Gives a script simple options (<code>--fps 24 --out review/</code>).
PyInstaller	Turns a Python tool into a double-click app for people who don't have Python.

Adding a UI: the ladder

Need	Use
Just run it	A Python script (+ Typer for options)
Quick UI, mostly for you	Gradio : a few lines give you a browser page with upload boxes, sliders and previews
A native window, or a panel inside a DCC	PySide6
A polished app to share with your team	Tauri (section 3)

Gradio's share link is public. `share=True` creates a link anyone can use, running on your machine and spending your API credits, for about a week. Keep it off unless you mean it.

PySide6 vs PyQt: nearly identical to write. Prefer **PySide6**: it's what Maya, Nuke, Houdini and 3ds Max ship, so tools move into a DCC easily, and its license (LGPL) is looser than PyQt's (GPL or paid).

Build a small local Gradio app with uv: I drop in a video, it shows the specs and gives me a button to make an H.264 review copy. Keep share off.

2. Tools inside your DCC

- **Scripts:** every app can run them today. See the Companion Guide's "Scripts for your apps" table for the language and where to paste.
- **Panels:** Maya, Nuke, Houdini and 3ds Max ship PySide, so ask for a docked panel with buttons. Blender uses its own UI system (`bpy` panels).
- **Real example:** one prompt ("write a Maya Python script that builds a copper material for Arnold from scratch") produced a script that built the whole shading network in Maya, and it looked just like copper.
- **One script, two ways:** the same Python file runs in the app's script editor, or **headless** (no window) across many files or on another machine (`blender -b`, `mayapy`, `hython`, `nuke -t`, `c4dpy`). See the Companion Guide, "One script, two ways".
- **MCP:** lets the assistant drive the app directly. MCP for Blender is the one we use; community MCPs exist for other apps, so check where they come from before installing.

3. Desktop apps with Tauri

Tauri builds small, fast desktop apps using web-page technology for the UI and a fast core underneath.

- **Reuses the web stack:** React, Vite, Tailwind, shadcn/ui, Lucide icons, Google Fonts. Learn one UI stack for both web and desktop.
- **Small and fast** compared to Electron, with real access to files.
- **Can bundle ffmpeg** as a "sidecar" (a helper program shipped inside the app). Ideal for batch converters, review players, asset browsers.

Know before you start:

- **Setup is the biggest hurdle:** it needs the Rust toolchain, plus build tools (Windows: Microsoft C++ Build Tools and WebView2; Mac: Xcode Command Line Tools; Linux: a list of system packages). Use setup prompt 7.
- **The first build is slow** (several minutes). Later builds are fast.
- **Unsigned apps show a warning** (SmartScreen on Windows, Gatekeeper on Mac) when you share them. Fine for in-house tools; tell people to expect it.
- **API keys still can't go inside.** A desktop app ships to other people's machines, and keys inside it can be extracted. Same rule as web apps.

4. Web apps

The frontend (what people see)

Piece	What it is
React	The most common way to build web UIs. Agents know it very well.
Vite	The build tool: fast previews while you work.
TypeScript	JavaScript with checks. Agents make fewer mistakes with it.
Tailwind CSS	Styling written right on the elements.
shadcn/ui	Good-looking ready-made components (buttons, dialogs, tables).
Lucide	A clean icon set.
Google Fonts	Free typefaces.

Hosting the frontend

- **GitHub Pages:** free; with GitHub Pro you can publish from a *private* repo. **The published site is still public;** only the code is private.
- **Netlify** or **Vercel:** also easy, with previews for every change and built-in serverless functions.

5. Backends and API keys

The restaurant. The **frontend** is the dining room: everyone walks through it, and anyone can look inside the code. The **backend** is the kitchen: staff only. An **API key** is the restaurant's credit card with its suppliers. You don't leave it on a dining-room table. **Full stack** is the whole restaurant: frontend, backend and database, built as one project.

Frontend asks. Backend spends.

An **API** is an ordering window another service opens for programs: send a request, get data or a result back. An **API key** is your account card at that window. It identifies you and usually gets billed.

Two kinds of backend:

- **Just hiding a key?** Use a **serverless function**: a tiny backend with no server to manage. Netlify, Vercel, Cloudflare Workers and Supabase Edge Functions all offer them.

- **Real app logic, scheduled jobs, long tasks?** Use **Railway**: it runs a full backend for you. Pick one language: **TypeScript** (Hono or Express) to match the frontend, or **Python** (FastAPI) if you know Python from your DCC.

Key hygiene

- Keys live in a `.env` file locally (blocked by the starter-pack `.gitignore`), and in the hosting platform's **environment variables** setting when deployed. Never in code, never in chat.
- **Web apps**: a variable starting with `VITE_` gets built into the page, so anyone can read it. Never put a key in one.
- **One key per project**, so you can switch one off without breaking others.
- **Set a spending limit** on every account.
- **Leaked a key?** Revoke it and make a new one. Don't just delete the file.
- **Tip**: paste the API's documentation link into the assistant and say *"use this"*.

APIs worth knowing

API	Good for
Weather, stock prices	Cheap or free. Perfect first API projects.
fal.ai	Image and video generation, upscaling, background removal, depth maps, segmentation. The VFX hook.
AI model APIs (Claude, OpenAI, Gemini)	Adding AI features to your own tools.

6. Data and storage

Supabase gives you a database, logins (auth) and file storage in one place, and the assistant can set it up through its CLI/MCP.

- **Publishable key** (`sb_publishable_...`): can be in the frontend **only** if Row Level Security is switched on for your tables.
- **Secret key** (`sb_secret_...`): backend only, ever.
- Older projects have "anon" and "service_role" keys; these are being retired by the end of 2026. If a key starts with `eyJ`, it's an old one.

Lots of media? Supabase storage gets expensive at volume. Use **Cloudflare R2** (no download fees) or **Backblaze B2** (very cheap storage) for video and image libraries.

7. Domains and DNS

A **domain** is your app's address (`mytool.com`). **DNS** is the settings that point that address at your app. DNS is tedious by hand and exactly what agents are good at, if your registrar lets them.

Registrar	Agent-friendly?
Cloudflare	Yes. It can register domains and manage DNS through its API and MCP.
DNSimple	Yes. Full API.
GoDaddy	Partly. Its official MCP only searches domains; the DNS API needs an account with at least one domain.
Squarespace Domains, Network Solutions	No DNS API.

Tip: you can register a domain anywhere and point its DNS (nameservers) at Cloudflare. Then even a Squarespace domain gets agent-friendly DNS without transferring it.

8. Let the agent check its own work

Ask it to use **Playwright** (or a browser MCP) to open your web app, click through it and take screenshots. That's the "prove it" habit, automated.

9. Later, when you need it

Stripe (payments) · **Resend** (sending email) · **Sentry** (error tracking). No need to learn these until a project calls for them.

Starter prompts

Create a new web app with React, Vite, TypeScript, Tailwind, shadcn/ui and Lucide icons, in a private GitHub repo. Explain the folder structure in 5 lines.

Add a serverless function that calls the weather API with a key from the environment, so the key never reaches the browser. Tell me where to put the key; don't ask me to paste it here.

Build a local Gradio app that sends a prompt to fal.ai and shows the image. Read FAL_KEY from a .env file. Check fal.ai's current docs first.

Create a Tauri desktop app with React, Vite, Tailwind and shadcn/ui that lists the videos in a folder I pick and shows their specs. Bundle ffmpeg with it.

Set up DNS on Cloudflare so <mytool.com> points at my Netlify site. Show me the records you'll create before you create them.